

Solving a Complex Language Game by Using Knowledge-Based Word Associations Discovery

Pierpaolo Basile, Marco de Gemmis, Pasquale Lops, and Giovanni Semeraro, *Member, IEEE*

Abstract—“The Guillotine” is a language game whose goal is to predict the unique word that is linked in some way to five words given as clues, generally unrelated to each other. The ability of the human player to find the solution depends on the richness of her cultural background. We designed an artificial player for that game, based on a large knowledge repository built by exploiting several sources available on the web, such as Wikipedia, that provide the system with the cultural and linguistic background needed to understand clues. The “brain” of the system is a spreading activation algorithm that starts processing clues, finds associations between them and words within the knowledge repository, and computes a list of candidate solutions. In this paper we focus on the problem of finding the most promising candidate solution to be provided as the final answer. We improved the spreading algorithm by means of two strategies for finding associations also between candidate solutions and clues. Those strategies allow bidirectional reasoning and select the candidate solution which is the most connected with the clues. Experiments show that the performance of the system is comparable to that of average human players.

Index Terms—Knowledge engineering, language games, natural language processing, spreading activation.

I. LANGUAGE GAMES

ONE of the most challenging fields in which Artificial Intelligence techniques have been exploited is represented by games. In terms of knowledge prerequisites it is possible to clearly distinguish closed-world games from open-world ones. Closed-world games provide the agent with all the knowledge necessary for playing the game, while for other kind of games it is not possible to completely define game play by means of given sets of rules. An interesting case of open-world games is represented by *language games*, in which word (and phrase) meanings play an important role [1]. In this paper, the performance of an artificial player at one such language game is drastically improved by ranking potential solutions from a spreading activation network effectively. Language games draw their challenge and excitement from the richness and ambiguity of natural language, and therefore have attracted the attention of researchers in the fields of Artificial Intelligence and Natural Language Processing.

Manuscript received September 17, 2013; revised April 17, 2014, and June 19, 2014; accepted August 30, 2014. Date of publication September 16, 2014; date of current version March 15, 2016.

The authors are with the University of Bari Aldo Moro, Bari 70121, Italy (e-mail: pierpaolo.basile@uniba.it; marco.degemmis@uniba.it; pasquale.lops@uniba.it; giovanni.semeraro@uniba.it).

Color versions of one or more of the figures in this paper are available online at <http://ieeexplore.ieee.org>.

Digital Object Identifier 10.1109/TCIAIG.2014.2355859

An example of a language game is *Who Wants to be a Millionaire?*, in which the player must have common sense knowledge or knowledge on the popular culture in order to answer a series of multiple-choice trivia questions. It has been shown that a system able to mine answers from the web plays the game about as well as people [2]. Other authors built a virtual player for that game, able to challenge human players, that incorporates common sense human knowledge extracted from Wikipedia articles by using Question Answering techniques [3]. Another remarkable example of a language game is *Jeopardy!*TM, an American quiz show demanding knowledge and quick recall, covering different topics such as history, politics, pop culture, science. *Jeopardy!*TM has a unique *answer-and-question* format in which contestants are presented with clues in the form of answers, and must phrase their responses in the form of a question. *Jeopardy!*TM poses a grand challenge for a computing system due to the variety of subject matter, and because the clues given to contestants involve analyzing complexities at which humans excel and computers traditionally do not. IBM WatsonTM is a question answering system which successfully challenged human champions of this game [4]. The sources for WatsonTM include unstructured content (encyclopedias, dictionaries, thesauri) as well as semistructured and structured content, such as databases, taxonomies, and ontologies (DBpedia, WordNet, Yago). All the data are mined to discover the semantically related but hidden concepts in a game question [5]. Another popular language game is solving crossword puzzles. The first experience reported in the literature is *Proverb* [6], that exploits large libraries of clues and solutions to past crossword puzzles. *WebCrow*¹ is the first solver for Italian crosswords. It exploits two main sources: web documents and previously solved games [7]–[9].

This paper studies “The Guillotine,” a challenging language game which demands knowledge covering a broad range of topics. The game, broadcast by Italian National TV, involves a single player, who is given a set of five words—the *clues*—each linked in some way to a specific word that represents the unique solution of the game. Words are unrelated to each other, but each of them has a hidden association with the solution. Once the clues are given, the player has one minute to find the solution. For example, given the five clues: *sin*, *Newton*, *doctor*, *New York*, *bad*, the solution is *apple*, because: the apple is the symbol of original *sin* in Christian theology; *Newton* discovered the gravity by means of an apple; “an apple a day keeps the *doctor* away” is a famous proverb; *New York* city is also called “the big apple”; and “one *bad* apple can spoil the whole bunch”

¹<http://en.wikipedia.org/wiki/WebCrow>

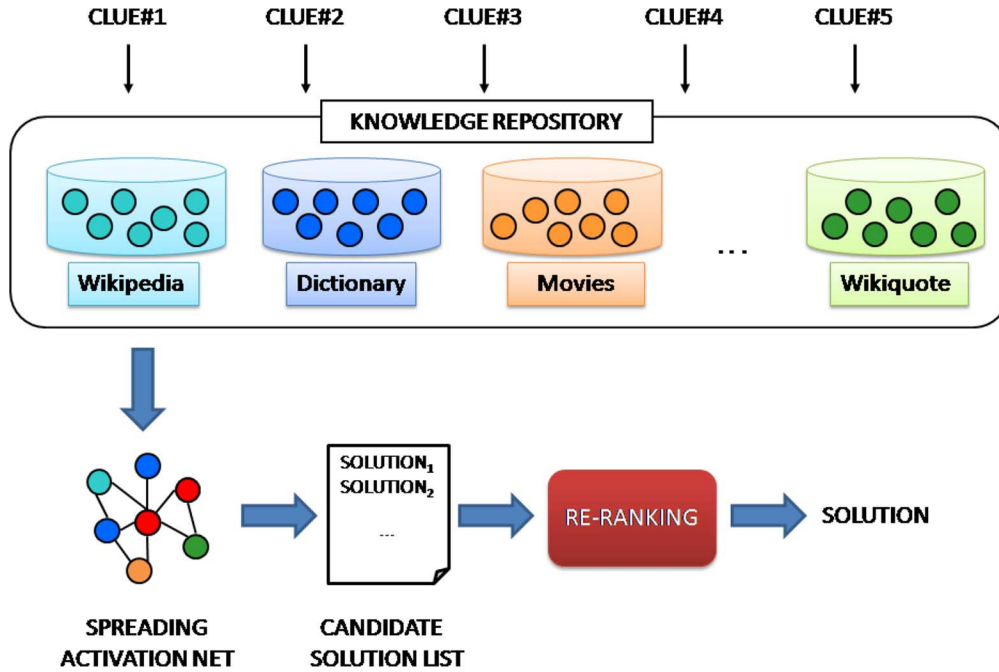


Fig. 1. The whole process implemented by OTTHO to find the solution for “The Guillotine” game. It starts from 5 clues and selects related *concepts*, that are exploited by the reasoning mechanism—based on a spreading activation network—for finding candidate solutions. Finally, the unique solution is selected by reranking algorithms.

is a popular phrase which figuratively means that the person doing wrong can have a negative influence on those around him.

In order to find the solution for a game, the human player must perform a complex memory retrieval task on her knowledge concerning the meanings of thousands of words and their contextual relations. No fixed sets of rules are sufficient to define the game play: the ability to find the solution depends exclusively on the background knowledge of the player and on her capability of interpreting and reasoning on the clues. Motivated by the challenging word association discovery task that the game issues, we developed an artificial player which exploits a spreading activation algorithm to correlate clues with words retrieved from a large knowledge repository extracted from the web. Results of our previous experiments showed that the knowledge on which the artificial player is based allows it to find multiple candidate solutions consistent with the provided clues, as well as that the hard part of the game is to discover the unique solution among them [10], [11].

The main contributions of this paper are two algorithms which improve the ranking of candidate solutions computed by the spreading algorithm proposed in [10], making the selection of the most promising one easier for the artificial player. In the following section, we describe the architecture of the system and briefly analyze the main limitation of the previously adopted reasoning mechanism. Section III presents the novel ranking algorithms, while Section IV describes new experiments performed to evaluate the system’s ability to solve the game.

II. AN ARTIFICIAL PLAYER FOR “THE GUILLOTINE”

OTTHO (On the Tip of my THOught) [10], [11] is a system designed to solve “The Guillotine” game, according to the

process depicted in Fig. 1. The core of the artificial player is the knowledge repository, which stores a background of general linguistic competencies and of more specific world facts exploited to discover hidden associations between words. It is automatically acquired from some selected sources available on the web, such as Wikipedia, Wikiquote, dictionaries. Clues are used to query that repository to retrieve the “pieces of knowledge,” i.e., *concepts*, which are most related to them. The retrieved concepts are then organized in a network in which a spreading activation algorithm searches for candidate solutions. The unique answer is provided by algorithms that rank candidate solutions according to some heuristics and select the most promising one among them (Fig. 1). In the following subsections we provide further details about the knowledge repository, the reasoning mechanism, and the novel algorithms for ranking candidate solutions.

A. The Knowledge Repository

The cultural and linguistic background of the artificial player is built by extracting information from textual sources on the web. The problem of turning automatically human-understandable content into a machine-understandable knowledge base has been deeply studied in machine learning literature [12]. Currently, popular and effective techniques are mostly statistical methods used to group, categorize, analyze, and retrieve documents [13]. Latent Semantic Analysis (LSA) [14] and other factor-analytic methods [15] have proven valuable for web content mining. One of the most recent lines of research, aiming at including knowledge into systems that perform tasks requiring human-level intelligence, is to mine Wikipedia pages. The approach proposed by Gabrilovich and Markovich [16], [17] employs machine learning techniques to provide a semantic repre-

sensation of a text document as a weighted vector of concepts, each concept corresponding to a Wikipedia article. The authors named their approach *Explicit Semantic Analysis (ESA)*, as opposed to LSA, since they use concepts that are explicitly defined by users. Other approaches extract explicit facts from *semistructured* components of Wikipedia, such as tables, because they are easier to process than plain text. Among these, the DBpedia project is the most prominent one [18].

The knowledge extraction process for OTTHO was mainly inspired by emerging approaches which integrate knowledge from Wikipedia with other linguistic resources, such as lexicons. A recent example of that line of research is YAGO, a large ontology automatically derived from Wikipedia and WordNet [19]. In fact, the actual issue here is how to integrate relevant information coming from different sources. In this work the following sources were used:

- **Encyclopedia**—the 2012 Italian Wikipedia dump²;
- **Dictionary**—the De Mauro Paravia Italian on-line dictionary (no longer available);
- **Compound forms**—groups of words that often go together having a specific meaning, crawled from the IntraText Digital Library³ and the on-line dictionary TLIO—Tesoro della Lingua Italiana delle Origini⁴;
- **Proverbs and Aphorisms**—the Italian version of Wikiquote;
- **Movies**—descriptions of Italian movies crawled from the Internet Movie Database⁵;
- **Songs**—Italian songs crawled from OnlyLyrics⁶;
- **Books**—book titles crawled from several web sites.

The heterogeneity of the knowledge sources involved in the process requires:

- the identification of the *basic unit* representing a *concept* in each specific source;
- the choice of a *unique* representation model to be adopted for all the basic units in the different sources.

As for the first issue, the idea is that the basic unit of a knowledge source corresponds to a *primary concept* it provides. Therefore, each source can be seen as a repository of basic units. As an example, let us consider a generic dictionary. Generally, an entry in this source corresponds to a word along with a bundle of usage information: definitions associated with its possible meanings, synonyms, examples of use, etc. Among all the information associated with a word, we define as the basic unit the short text providing the meaning for that word, along with its synonyms. In case of polysemous words, a basic unit is defined for each meaning. Fig. 2 depicts the entry for the word “sin.” The basic unit is highlighted in bold.

The other issue is the adoption of a model for the *harmonization* of basic units belonging to different sources and their representation in a *homogeneous* way. Since information in each source is stored in textual format, and since in practice each basic unit is a fragment of text, we decided to adopt the bag-

```
[sin (noun) | def.: a transgression
against divine law.
synonyms: immoral act, act of evil.
examples: "a sin in the eyes of God",
"the human capacity for sin".]
```

Fig. 2. The entry for the word “sin” in the Dictionary source (translated into English). The short text that gives the definition, along with the synonyms of “sin”, is the basic unit for that entry (highlighted in bold).

```
[CU_24 | sin 1.54
transgression 0.54
divine 0.45
law 0.44]
```

Fig. 3. The CU for “sin” in the Dictionary source. The CU is uniquely determined by its identifier and contains keywords and their tf-idf scores in the Dictionary corpus.

of-words (BOW) model, a well-known representation used in Natural Language Processing (NLP) and Information Retrieval. In this model, a text is represented as the bag (multiset) of its words, disregarding grammar and even word order, but keeping multiplicity.

According to the adopted model, each source is a corpus of documents, i.e., text fragments corresponding to basic units, represented as BOWs. “The term weighting scheme we adopt, tf-idf (short for term frequency-inverse document frequency), is a numerical statistic that takes into account word occurrences in the document, as well as frequency of words in the whole corpus [20].

We call the BOW representation of a basic unit a *Cognitive Unit* (CU), because it provides the machine-readable format of the concepts on which the reasoning mechanism works. The name stems from the Adaptive Control of Thought (ACT) theory by J. R. Anderson, according to which information in the long term memory of human beings is encoded as cognitive units that form an interconnected network [21]. The reasoning algorithm of the system is inspired by that theory (Section II-B). Fig. 3 depicts the CU corresponding to the basic unit described in Fig. 2.

Some NLP operations are applied on basic units to obtain the corresponding CUs [22]. Table I provides the description of the basic unit chosen for each source, as well as the specific NLP operations applied on the text to turn basic units into CUs.

Each individual knowledge source is turned into a separate CU repository. The whole knowledge repository (KR) of the system is the set of CU repositories associated with the 7 sources listed in Table I, which contains a total of 743 192 CUs.

The main advantage of having CUs in the form of BOWs is that CU repositories can be represented by using the Vector Space Model, widely adopted in Information Retrieval [23]. This provides an easy and immediate way to find relevant CUs associated with clues, by simply querying the CU repositories with clues and computing relevance as cosine similarity between them and all CUs (Fig. 1). Most relevant CUs are then exploited by the reasoning mechanism, as described in the next section.

²dumps.wikimedia.org/itwiki/

³www.intratext.com/bsi/listapolirematiche/indalfa.htm

⁴ovipc44.oiv.cnr.it/Tliopoli/

⁵www.imdb.com

⁶www.onlylyrics.com/

TABLE I
DEFINITION OF BASIC UNITS AND COGNITIVE UNITS IN EACH KNOWLEDGE SOURCE. LEGEND FOR NLP
OPERATIONS: TOK = Tokenization; STP = stopword elimination; LEM = lemmatization;
NER = named entity recognition; HEU = Heuristics Adopted for Boosting tf-idf Scores of Emphasized Words;
SEL = Feature Selection by Using tf-idf Thresholding to Detect Most Informative Words

Knowledge source(s)	Basic Unit	Cognitive Unit
Encyclopedia	Wikipedia article	keywords in the title and most informative keywords within the article. NLP operations: TOK, STP, LEM, NER, HEU, SEL
Dictionary	Meaning of a word + synonyms	keyword, its synonyms, keywords in the definition NLP operations: TOK, STP, LEM, HEU
Compound forms	Entire compound form	All the keywords in the compound form. NLP operations: TOK, STP, LEM
Proverbs and Aphorisms	Single quote	keywords in the quote and name of the author. NLP operations: TOK, STP, LEM, NER
Movies	Title and Actor(s)	keywords in the title and name of actor(s). NLP operations: TOK, STP, LEM, NER
Songs	Title + Player(s)	keywords in the title and name of player(s). NLP operations: TOK, STP, LEM, NER
Books	Title + Author(s)	keywords in the title and name of the author(s). NLP operations: TOK, STP, LEM, NER

[CU_2 | sin 1.93

Christianity 1.62

apple 1.45

Adam 1.65

Eve 1.64]

[CU_14 | apple 1.77

day 0.61

doctor 1.96

away 0.43]

Fig. 4. Another CU for “sin,” in addition to the one obtained from the Dictionary. It could be retrieved, for example, from Wikipedia.

[CU_8 | doctor 1.55

house 0.77

american 1.62

drama 1.45]

Fig. 5. Two CUs for “doctor.”

B. The Reasoning Process

Turning several human-readable knowledge sources into a machine-understandable repository of CUs allows some kind of “knowledge infusion” into the system. The goal of the reasoning process is to exploit the large amount of knowledge infused into OTTHO for connecting clues with related concepts (CUs) that can guide the system toward the solution.

As introduced in Section II-A, the reasoning mechanism is inspired by the ACT theory [21], according to which words and their meanings are stored in the mind in a network-like structure. Within such a network, the meaning of a word is represented by nodes as well as by their relations with other nearby nodes.

The algorithm is based on a *spreading activation model*, which consists of a network data structure of nodes interconnected by links, that may be labeled and/or weighted, and usually have directions [24]. This model has been extended to include both words and CUs in the network, so that a semantic memory of concept-based knowledge is available for the reasoning step, i.e., the search process over the network that aims at finding the most related words associated with clues.

Our method that builds the Spreading Activation Network (SAN) for a run of the game takes as input the clues $k_1, \dots, k_5$. Initially, source nodes $N_1, \dots, N_5$ are added to the network, each one labeled with the corresponding clue k_i . For each k_i , a search is performed in the knowledge repository KR defined in the previous section, and a list of relevant CUs C_j is retrieved. CUs are ranked according to their similarity score w_{ij} with the clue. Then, retrieved CUs are included in the SAN. Each C_j is linked to the corresponding source node k_i ; the edge is oriented from k_i to C_j and labeled with w_{ij} . In order to control the size of the SAN, we take the top- N ranked CUs related to clue k_i (different values for N are tested in the experiments reported in Section IV). At this stage of the SAN building process, edges represent associations between clues and CUs, while similarity values measure the strength of those relationships. Finally, for each CU node, word nodes labeled with terms in the description

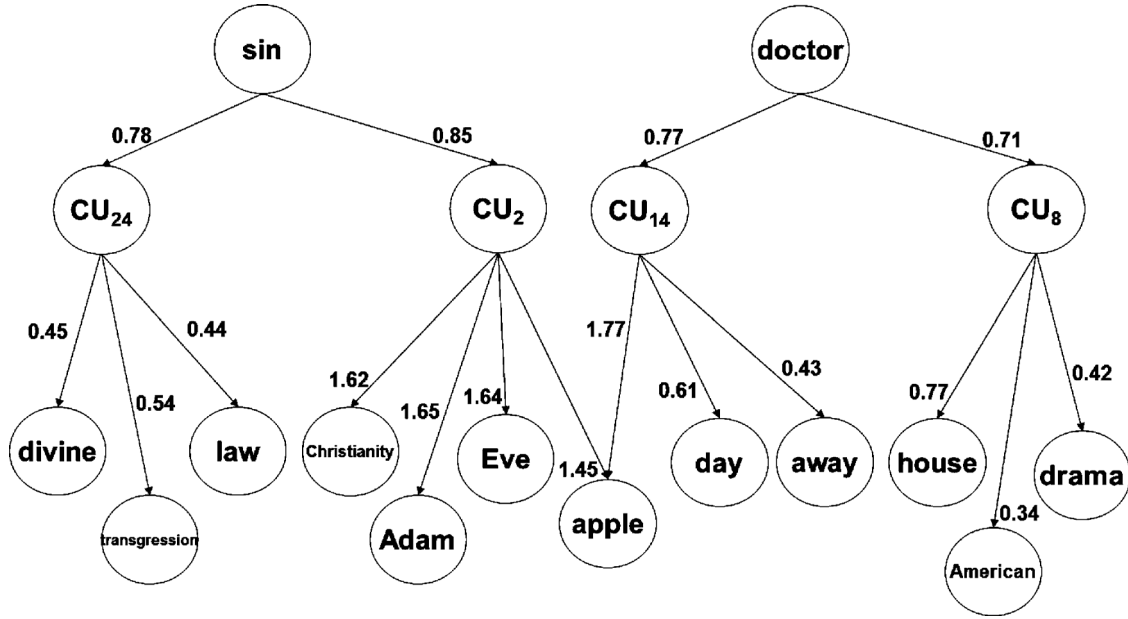


Fig. 6. SAN for the clues “sin” and “doctor.”

of the CU are included in the SAN. Links are created from the CU node towards its word nodes and labeled with tf-idf scores of words (Section II-A).

An illustrative example of a SAN, built for the two clues *sin* and *doctor* (for the sake of simplicity) is depicted in Fig. 6. Two CUs retrieved for “doctor” are reported in Fig. 5, while the two CUs found for “sin” are the ones reported in Figs. 3 and 4.

Once the SAN is built, the reasoning process starts from the clues, which trigger the search process over the network. Each node n_i has an associated *activation level* al_i , which is a real number in the range $[0.0 \dots 1.0]$, and represents the level of stimulus of the node. At time $tm = 1$ the SAN is initialized by setting all activation levels to 0, with exception of the clues, whose activation level is set to 1. A *threshold* F , a real number in the range $[0.0 \dots 1.0]$, determines if a node is fired, that is to say whether it can spread its activation level over the SAN. Every *fired* node propagates its own activation value to its neighbors as a function of both its current activation level and the weights of the edges that connect it with its neighbors, and a decay factor D that limits the propagation of the activation value through the network. The activation level of neighbors is updated accordingly. At time $tm = 2$, all clues are fired and the amount of activation spreading from them updates the activation level of CU nodes. At time $tm = 3$, only CU nodes whose activation level exceeds F are fired and propagate their activation values towards their neighbors, i.e., word nodes. The final activation value of these nodes allows to identify the most “active” words which potentially might be the solution of the game. In fact, all word nodes are ranked in descending order according to the activation value, and the corresponding words are included in the Candidate Solution List (CSL), which will be then processed by other algorithms, described in the following section, to determine the unique answer for the game.

The status of the SAN for our example after the spreading process is depicted in Fig. 7. Darker nodes are those with a

higher level of activation. The node *apple* is the one with the highest value since it is updated by two CU nodes. Therefore, *apple* is the best candidate solution, but *Adam*, *Eve*, *Christianity* and *house* are good potential solutions as well, because of the high weights on their incoming edges.

III. FINDING THE SOLUTION

The simplest approach to select the unique answer for a run of the game is to choose from the CSL the word corresponding to the node with the highest activation level. Previous experiments reported in [10] showed that the accuracy achieved by that strategy, measured as the percentage of solved over attempted games, does not exceed 10%. On the other hand, we observed that in 40% of attempted games the solution was found among the top-50 positions of the CSL. These results convinced us that the activation level of nodes can help to define an order among candidate solutions, but more sophisticated techniques are required to select the unique answer among them.

We propose two unsupervised approaches for improving the ranking computed by spreading activation:

- *Reverse Paths (RP)*—This strategy is based on the main rule of the game: the solution is the word which has a hidden association with *all* the clues. Therefore the idea is to rank candidate solutions according to the number of clues to which they are connected with in the SAN. More details will be provided in Section III-A.
- *Expanded Reverse Paths (ERP)*—This strategy is inspired by one possible way the human player searches for the solution: sometimes a subset of clues acts as a spark and leads the player towards a possible solution. Then, she tries to validate the hypothesis by searching for *new* connections between the candidate solution and the clues, that she had not thought of initially. Therefore, this approach also tries to find reverse paths from candidate solutions to clues, but the difference from the previous method is that it goes back

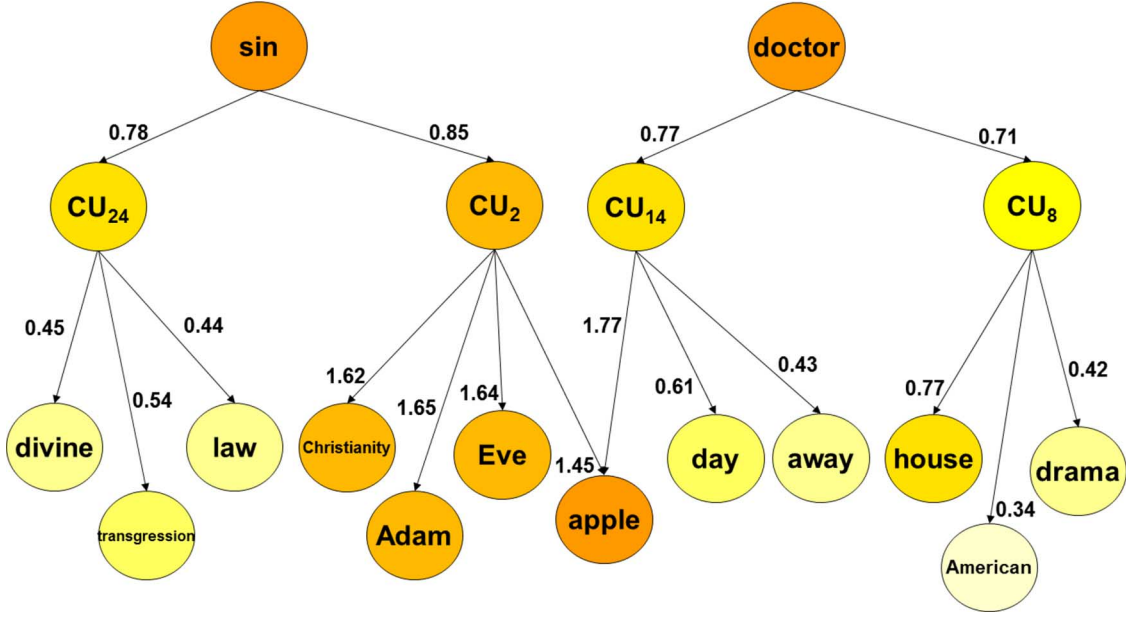


Fig. 7. SAN for the clues “sin” and “doctor” after the spreading process.

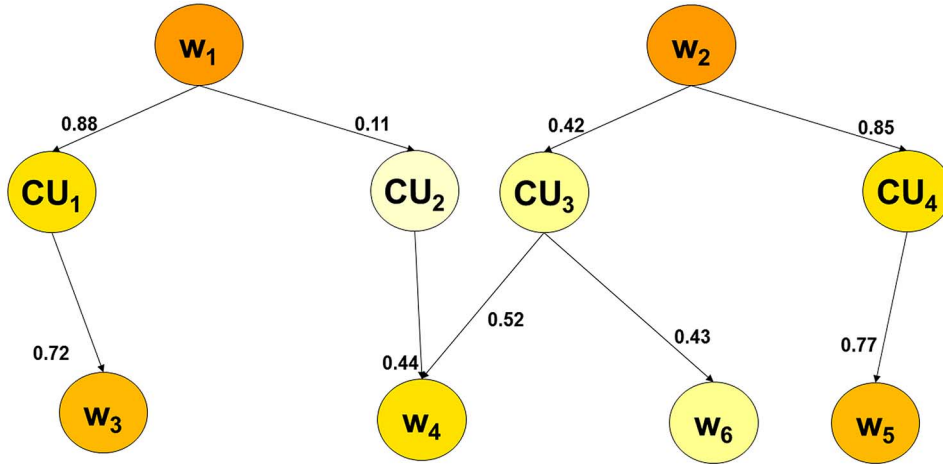


Fig. 8. Example of a candidate solution connected to more than one clue.

to the knowledge repository to find *new* nodes associated with candidate solutions, that are included in the original SAN once the spreading process ended. They are exploited to find *new paths* from candidate solutions to clues. More details will be provided in Section III-B.

A. Reverse Paths (RP)

This approach computes, for each word node w_i , the number of different clues $nclues_i$ connected to w_i by a path in the SAN. It simply counts the number of different clues reached by all paths starting from a candidate solution. Only for the computation of that score the edges are crossed in the opposite way. For example, in Fig. 8, $nclues_3 = 1$ since w_3 is connected to w_1 through CU_1 , while $nclues_4 = 2$ since w_4 is connected to w_1 through CU_2 and to w_2 through CU_3 . If words were ranked according to their activation level, w_3 and w_5 would be the top ranked candidate solutions, but if we take into account the number of clues to which candidate solutions are connected, w_4

might be a better solution and therefore it should be pushed up in the ranking.

The final ranking of candidate solution w_i is computed by taking into account both factors: $nclues_i$ and al_i , that is the activation level of w_i at the end of the spreading process. The ranking is defined as follows:

$$w_i > w_j \iff (nclues_i > nclues_j) \vee ((nclues_i = nclues_j) \wedge al_i > al_j). \quad (1)$$

The main criterion is the number of clues reached by reverse paths, and then, in case of tie, the activation level is adopted.

B. Expanded Reverse Paths (ERP)

The ranking of candidate solutions is defined by this approach according to (1), as for the previous one, but $nclues_i$ is computed on a new SAN obtained from the original one, built as reported in Section II-B. The strategy is described by

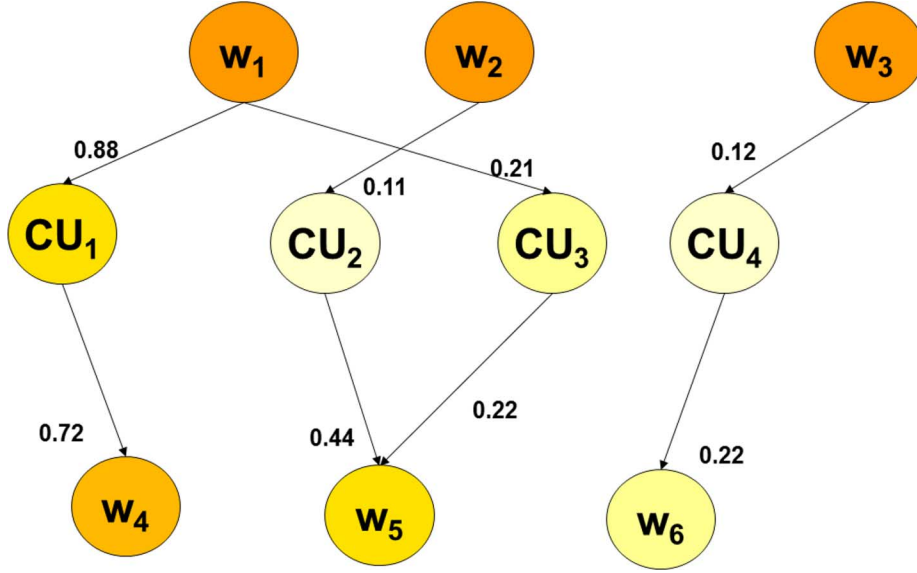


Fig. 9. The SAN after the spreading process: w_6 is connected only to clue w_3 through CU_4 and has the lowest activation value, so it is the worst candidate solution.

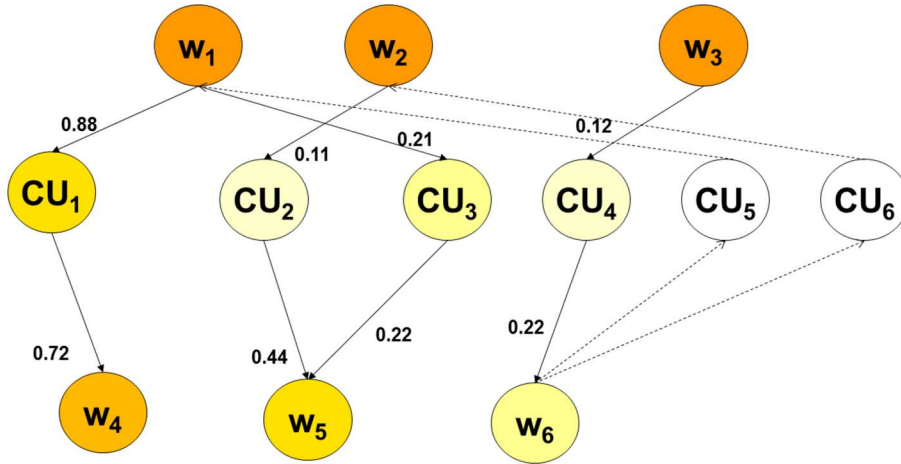


Fig. 10. SAN after running Expanded Reverse Paths. New nodes CU_5 and CU_6 are connected to w_6 through new dotted edges.

means of the example depicted in Fig. 9, which shows the status of the initial SAN after the spreading process, and Fig. 10, which shows the SAN updated by the *Expanded Reverse Paths* method.

Let us consider the candidate solution w_6 . Initially (Fig. 9), it was connected only to clue w_3 through CU_4 . If candidate solutions were ranked according to the activation level only, then w_4 would be the answer, while if *Reverse Paths* were adopted, the ranking of candidate solutions would be: w_5, w_4, w_6 . *Expanded Reverse Paths* uses candidate solutions to query the KR for finding relevant CUs in the same way as clues w_1, w_2, w_3 were used to retrieve $CU_1, \dots, CU_4$ when building the initial SAN. Retrieved CUs which do not occur in the initial SAN are included in the new SAN and connected through new edges to the words from which they were retrieved. Then, for each new CU node CU_i , the set of keywords W_i in the CU description is extracted and, if a clue w_k occurs in W_i , a link is created from CU_i to w_k .

In our example, let us suppose that any new CU is retrieved by w_4 and w_5 , while CU_5 and CU_6 are retrieved by w_6 and connected to the corresponding node.

New links are created from CU_5 towards w_1 and from CU_6 towards w_2 . In other words, the SAN is updated to include two new concepts recalled by w_6 and the corresponding connections with clues. The value of $nclues_6$ computed after updating the SAN is 3 (it would have been 1 by using the *Reverse Paths* approach). The new ranking of candidate solutions is: w_6, w_5, w_4 .

IV. EXPERIMENTAL EVALUATION

The goal of the experiments is to evaluate the effectiveness of the reranking methods proposed in the previous section in solving the game. Since the effectiveness depends on the reasoning algorithm, as well as on the knowledge repository it works on, those methods are compared to alternatives that exploit knowledge sources other than those used to build the CU repository.

A. Datasets and Measures

We used two datasets in our experiments:

- TV_GAME, which includes 266 guillotines, i.e., sets of 5 clues, attempted by human players during the TV show, along with their correct answers⁷;
- BOARD_GAME, which includes 150 guillotines and correct answers contained in the official board game, which resembles the TV game.

As for the performance measures, *Precision* (P) provides the accuracy of the method. It is computed as the ratio between solved games, the guillotines for which the method finds the correct answer, and attempted games

$$P = \frac{\#SOLVED_GAMES}{N}. \quad (2)$$

Another measure, *Precision@k* ($P@k$), estimates the accuracy whether a CSL is provided instead of a unique answer. It is computed as

$$P@k = \frac{\#K-SOLVED_GAMES}{N}. \quad (3)$$

We call a game *k-solved* by a method which computes a CSL if the correct answer occurs among the top-k ranked candidate solutions in that list. By varying k , this measure gives a qualitative assessment of the candidate solutions computed by the method, even if it does not provide a unique answer.

B. Methods Compared to Our Approach

1) *Baseline: Pure Spreading Activation Algorithm*: The baseline for our experiments is the “pure” spreading activation algorithm described in Section II-B, with no methods for reranking. The answer given by this method is the word corresponding to the node with the highest activation level at termination time. This is the basic approach adopted in the previous version of the system [10].

Two parameters control the size of the SAN:

- h , the maximum number of relevant CUs retrieved by querying a knowledge source with a clue;
- t , the threshold for tf-idf associated with words in CUs.

Several values are tested for h : 1, 5, 10, 20, 30. For $h > 10$, the precision of the system decreases (and furthermore the large size of the SAN does not allow the system to provide the answer in 1 minute, as the rules of the game require). For this reason, in Section IV-C we present the results only for $h = 1, 5, 10$ and provide a possible explanation for the degradation of performance at higher h values. Parameter t is used to prune the SAN by neglecting the less significant words. All word nodes whose tf-idf is lower than t are deleted. Values vary between 0 and 0.5 in order to limit the aggressivity of the feature selection. Values for spreading activation parameters F (firing threshold) and D (decay factor) are experimentally tuned, since no general setting is provided in literature.

2) *Query Expansion*: Solving the guillotine game has been recognized in IR literature as a query expansion problem [25]. The problem consists in expanding a seed query q with additional terms that improve retrieval performance. The output of

the process is a new query formed by an expanded set of terms with their associated weights, usually computed by exploiting a source of data (e.g., a corpus of documents). According to the approach suggested in [26], the expansion terms are the most relevant ones extracted from the top-ranked documents in the result set for q . Relevance of terms is computed by a term-ranking function.

We follow this approach to define a method that computes candidate solutions as a set of expansion terms. The seed query q is the disjunction of the five clues, while the data source is the whole CU repository KR .

The terms included in the CSL are those contained in the CUs retrieved in response to q from KR , and ranked by the Kullback-Leibler distance, defined as

$$p(t | R) \cdot \log \frac{p(t | R)}{p(t | KR)} \quad (4)$$

where $p(t | R)$ and $p(t | KR)$ are the probability of occurrence of term t in the set of the top-ranked documents R and in the whole CU repository KR , respectively.

Probability $p(t | R)$ is computed according to the maximum likelihood criterion, as the ratio between the number of occurrences of t in R , treated as a long sequence of terms (i.e., as a single document), and the total number of terms in R . Probability $p(t | KR)$ is computed in the same way, but term occurrences are counted in KR instead of R .

The first term in the CSL is the answer provided. We evaluate the precision of this method when the result set for q is limited to 1, 5 or 10 CUs, in order to ground the computation on the same amount of knowledge exploited by other compared methods, such as the pure spreading activation algorithm.

3) *Freebase CSL*: We propose a simple algorithm that computes a CSL by relying only on the knowledge coming from Freebase⁸, a community-curated knowledge base of data harvested from many sources, which contains *entities*, such as well-known people, places, and things. The algorithm is an alternative to our approach based on CUs that we want to evaluate in order to understand whether the solution of the game can be reached in a simpler and less computationally costly way. In fact, APIs that allow easy and quick access to Freebase information are provided by Google⁹, whose Knowledge Graph is partially built upon Freebase.

Freebase is a graph database, so its data structure is defined as a set of nodes and a set of links that establish relationships between the nodes. The entity (called also topic) is the fundamental unit in Freebase. It represents a single concept or real-world thing, and is represented by a node in the graph. Examples of entities are: *Alan Turing*, *Computer Science*, the *Battle of Stalingrad*. As of April 2014, the database has approximately 43 million entities. Data stored about them depend on their *type*, which defines the particular kind of entity and denotes an IS-A relationship. A type is associated with properties, which define the unique qualities of that type. For example, the movie *E.T. the Extra-Terrestrial* (entity) is a film (type) directed by (property) Steven Spielberg (entity).

⁸<http://www.freebase.com>

⁹<https://developers.google.com/freebase>

⁷Manually collected by monitoring the TV show.


```

Entity#1
  label: Apple Inc.
  relevance score: 212.383942
  notable: Consumer electronics
           Business
Entity#2
  label: Apple Store
  relevance score: 68.498772
  notable: Computer hardware
           Business

Terms included in CSL:
Business      280.882714
Consumer      212.383942
Electronic    212.383942
Computer      68.498772
hardware      68.498772

```

Fig. 11. Two entities retrieved by Freebase Search. Only *label*, *relevance score*, and *notable* are reported; other properties are omitted for brevity. The weight assigned to “Business” is the sum of the relevance scores of both entities.

We use the Freebase Search API that, given a free text query, retrieves a result set of entities, ranked by a relevance score (computed by the API as well) that takes into account the number of both inbound and outbound links. The algorithm proceeds as follows. One query q_i for each clue k_i of a game is performed and a result set R_i is obtained. Given an entity e_j in R_i , we extract the keywords from the properties *label*, the name of the entity, and *notable*, the short text that qualifies the entity, and include them in the CSL (k_i is excluded from the list since clues cannot be solutions). Fig. 11 shows two entities retrieved for the query *apple* and the corresponding terms included in the CSL. The weight assigned to a term is the sum of the relevance scores associated to retrieved entities in which the term occurs.

Keywords and weights in the CSL are updated until all the 5 result sets have been processed, and the term with the highest weight is proposed as the solution. In order to compare all alternative methods under the same conditions, we evaluate the performance of this algorithm when 1, 5, or 10 entities are included in R_i .

4) *Hybrid CSL by Using Cognitive Units and Freebase*: We consider a “hybrid” competing algorithm, which exploits the knowledge stored in our CU repository to build the initial CSL, and then Freebase entities for reranking. The baseline method (Section IV-B-1) is used to compute the initial CSL, then the score of those candidate solutions which are connected to clues through Freebase entities are updated and a new ranking is computed, in the same way as the ERP algorithm does. Therefore, this methods can be seen as a direct alternative to the proposed ERP method. The details of the algorithm follow. For each candidate solution w_i , a query is performed by the Freebase Search

API and keywords are extracted from label and notable properties of retrieved entities. Then, we count $nclues_i$, the number of clues found among the keywords extracted. The final ranking of w_i is computed as in (1), by taking into account both $nclues_i$ and the activation level of w_i given by the baseline method. The first term in the updated CSL is proposed as the solution. Also this method is evaluated under the same conditions as the other alternatives, therefore we limited to 1, 5, or 10 the number of entities retrieved by the Freebase Search API for a candidate solution.

5) *Human Players*: We recruited 30 people according to the availability sampling strategy [27], in order to define the precision of the average human player. Each participant was required to solve 10 guillotines, 5 selected at random from each of the two datasets. In that way, 150 games were attempted by humans for each dataset. According to the rule of the game, the answer was provided within one minute from when the player were given the clues. The precision of each player was measured, like for systems, as the ratio between the number of solved guillotines and the number of attempts. The result, averaged across the 30 participants, provides us “an estimate of human accuracy” that can be compared with that of the proposed algorithms.

C. Results

The results of the comparison of RP, ERP and the baseline method that uses activation level (AL) to rank terms in the CSL are shown in Figs. 12 and 13. They report the precision P obtained by several runs of the experiment performed by varying the parameters that control the size of the SAN.

The main outcome is that both ERP and RP approaches for reranking the CSL outperform the AL baseline on both datasets. In particular, ERP performed better than RP, with a dramatic increase of precision with respect to the AL baseline: $P = 0.26$ (+438%) on TV_GAME and $P = 0.29$ (+238%) on BOARD_GAME, both obtained whether a high number of CUs is included in the SAN ($h = 10$) and feature selection is applied ($t = 0.4$). Indeed, this configuration of parameters causes the best outcome for both reranking methods. Under these conditions, very poor results are observed for the AL baseline, which lead us to attribute the improved precision to the new ranking computed by RP and ERP methods. On the other hand, decreasing performance is observed whether $h = 20$ or $h = 30$, perhaps meaning that a larger amount of knowledge without control might overload the system and led to generate noisy networks in which it is more difficult for the spreading process to rank the solutions first (or even include them in the CSL at all). A possible explanation for the drop of precision is that retrieved CUs are ranked in descending order by their similarity score with the clues, and included in the SAN accordingly. Therefore, an increased number of CUs implies that the SAN is extended with more information about the clues, but the relevance of this information (measured by the similarity score) decreases. If some less relevant CUs are fired, they propagate their activation values toward their words that might as a result be included in the CSL. This could happen especially for most common words, which occur in the BOW representation of many CUs.

The activation level of the nodes corresponding to those most

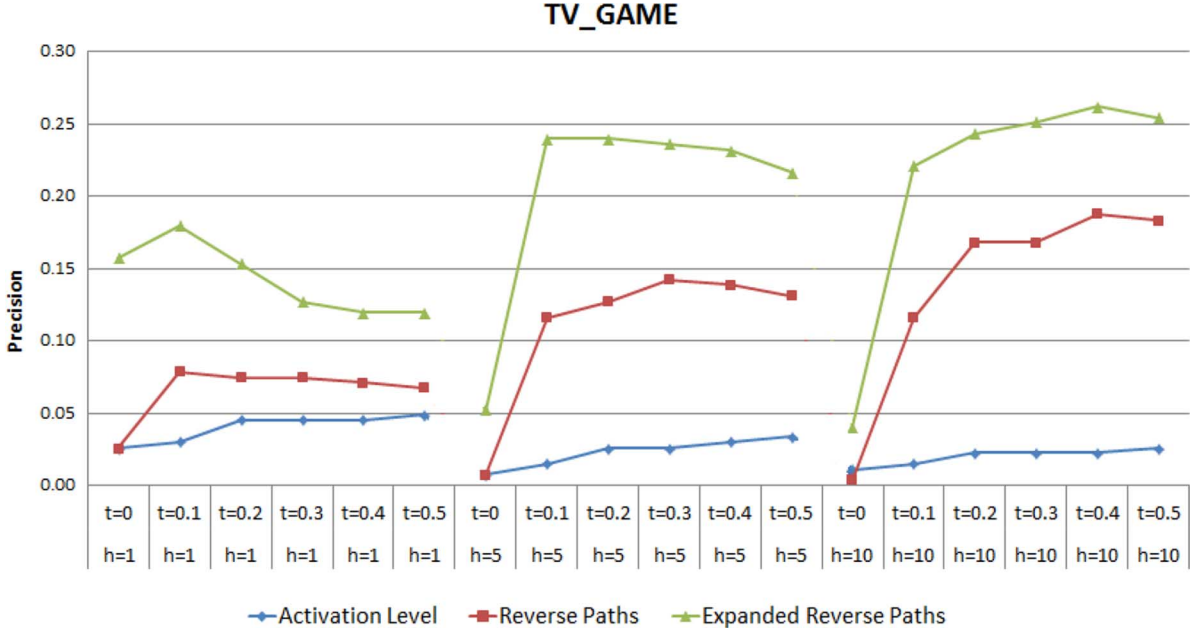


Fig. 12. Precision obtained by the two novel reranking methods on the TV_GAME dataset.

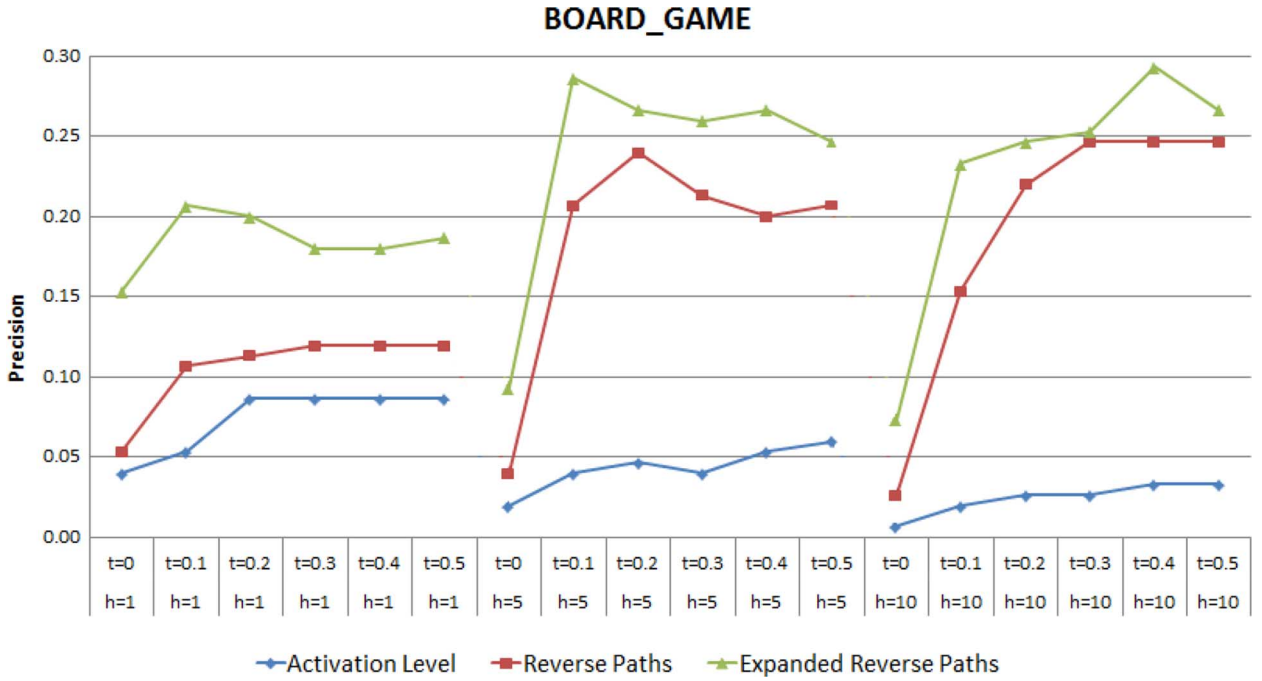


Fig. 13. Precision obtained by the two novel reranking methods on the BOARD_GAME dataset.

common words can be updated by many CUs and then they could be ranked highly in the CSL, with the risk that the actual solution is pushed down.

Furthermore, a significant drop of precision is observed for $h = 5$ or $h = 10$, when $t = 0$, which corresponds to including in the SAN *all* the words of selected CUs. Again this could be the consequence of including too much information in the SAN. These results confirm the relevant role of feature selection. The same trend is observed on both datasets. For brevity, we do not report the results for $h > 10$.

Fig. 14 shows the precision of all the compared methods on the two datasets. For methods whose precision depends on the

SAN parameters h and t , we reported the best outcome obtained by varying them as shown in the previous graphs.

In order to provide a confidence interval for the results, we modeled each game as a Bernoulli random variable, since we have only two possible outcomes, i.e., the provided answer for a guillotine is correct or incorrect. Given the probability of a success p (i.e., the precision of each method), the Bernoulli random variable has mean $\mu = p$ and standard deviation $\sigma = \sqrt{p(1-p)}$. With a sample of n independent trials, the standard error for the sample mean is $SE = (\sigma)/(\sqrt{n})$. By assuming that the distribution of the sample mean is well approximated by a normal model, we can build a 90% confidence

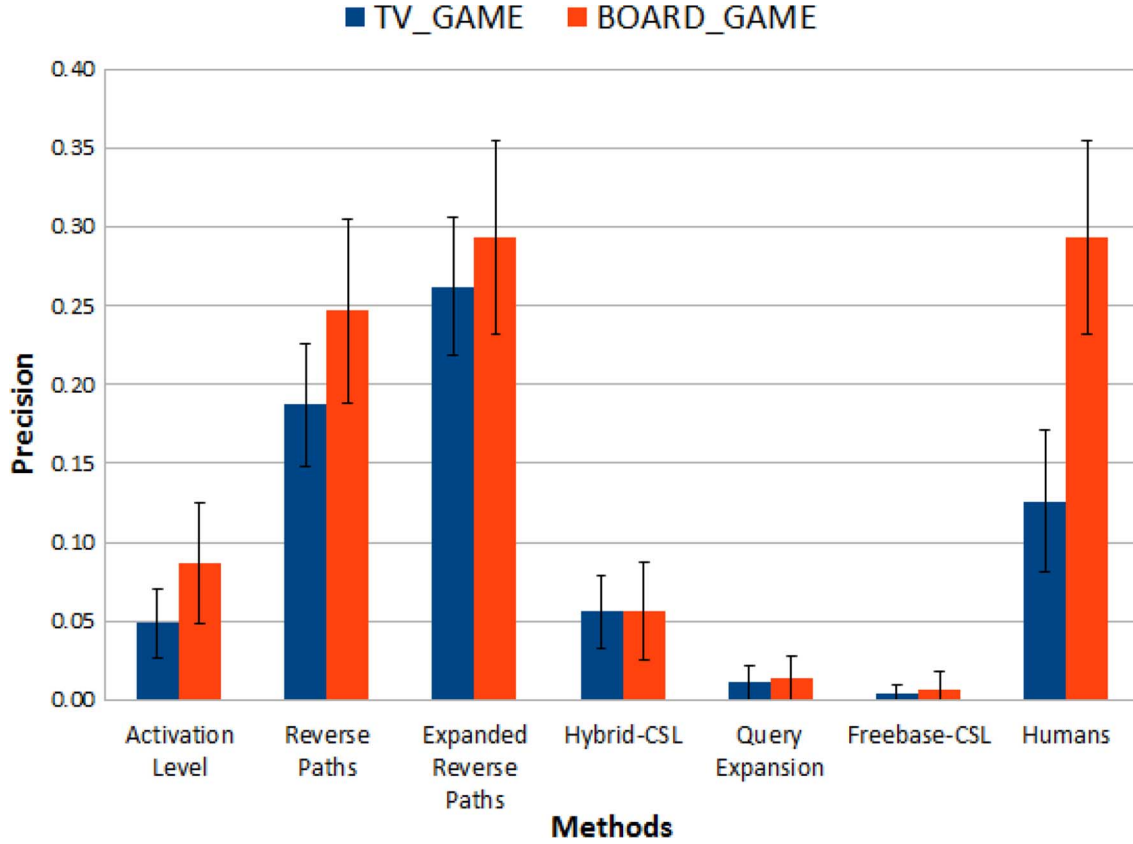


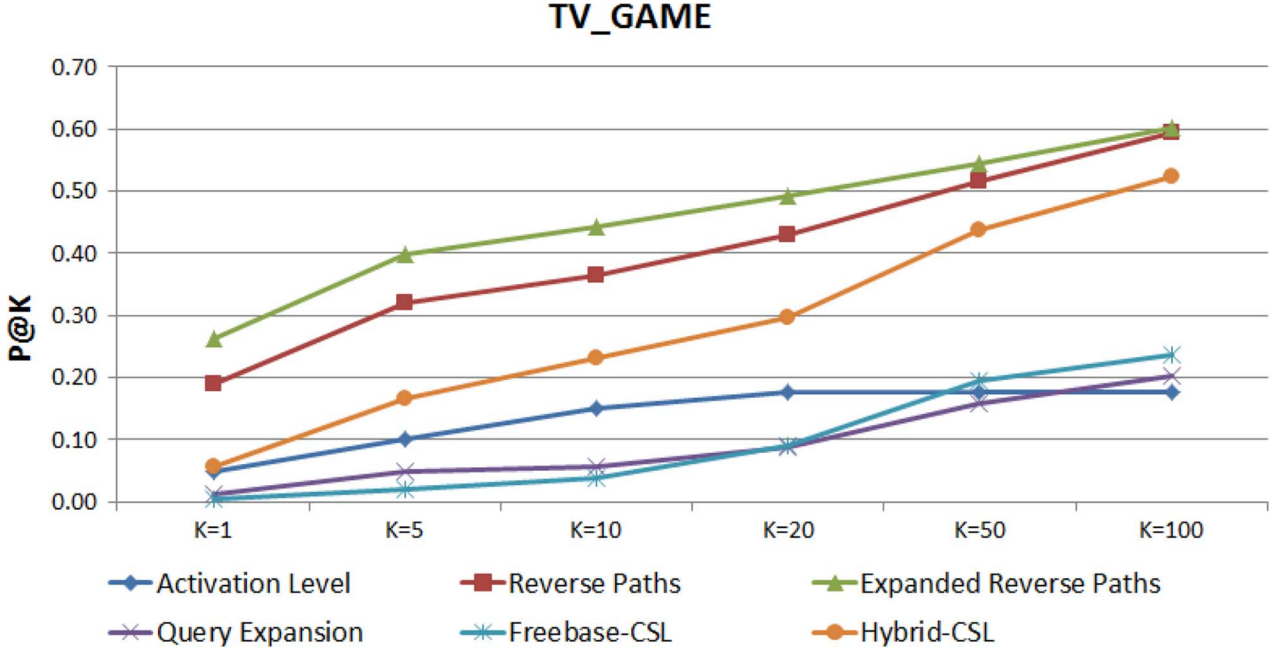
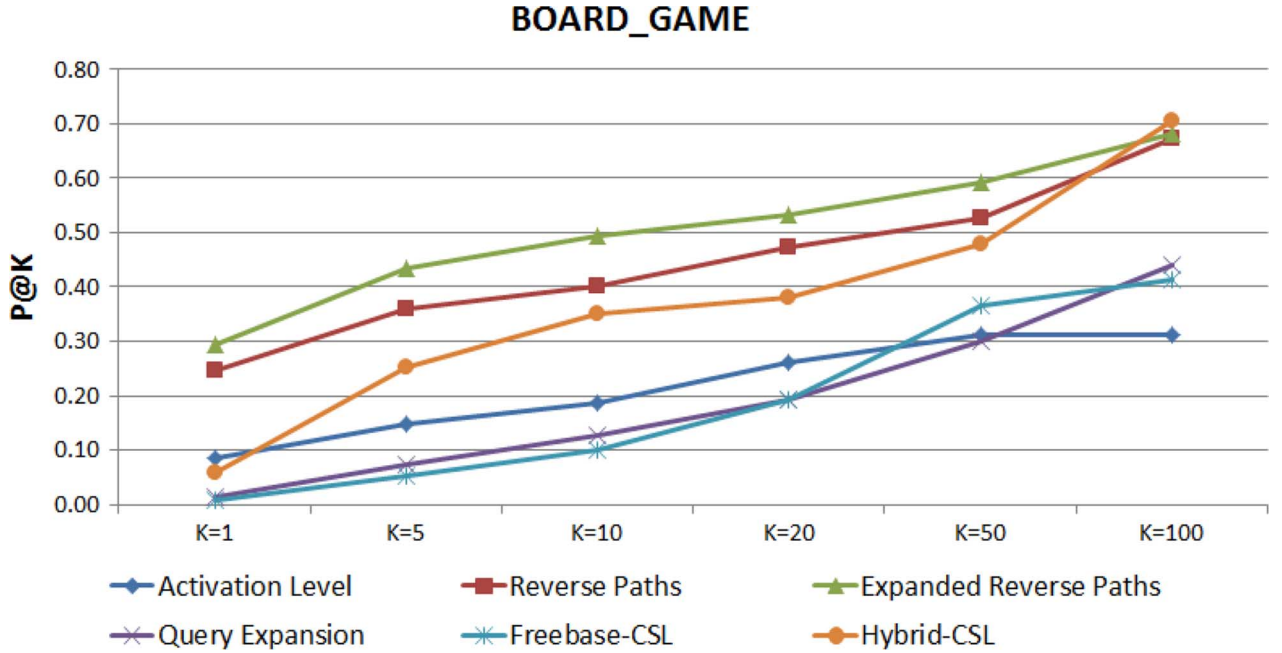
Fig. 14. Comparison of precision of all methods on both datasets. For methods whose precision depends on the SAN parameters h and t , we reported the best results obtained by varying them as shown in Figs. 12 and 13. Error bars indicate a 90% confidence interval for the precision values.

interval $p \pm 1.645 \times SE$ (we are 90% confident that the precision of each method falls in that interval).

The main outcome is that ERP outperformed all the other methods on both datasets. The performance is even better than that of human players on TV_GAME ($P = 0.126$), while on BOARD_GAME the difference is not statistically significant. In fact, the human precision increased up to $P = 0.293$ on BOARD_GAME. Higher human performance on BOARD_GAME could be due to the fact that guillotines in this dataset are meant just for fun, whereas those in TV_GAME are intended to challenge the contestants of the TV show. Therefore, it is quite understandable that they turn out to be more difficult for the average player.

Indeed, RP and ERP are the only methods that challenge human players on hard guillotines. One possible explanation for this result is that hard guillotines generally have very specific clues and require more extensive knowledge about world facts and particular topics to find the solution than the average player has. Conversely, specific clues are more likely useful for the system to retrieve the solution since they allow to perform discriminative selection of CUs in the large knowledge repository. As an example, consider the two clues *brother* and *computer*. It is difficult for the average human player to connect them to the word *sun*, while the two CUs related to the movie “*Brother Sun Sister Moon*” and to the company “*Sun Microsystems, Inc.*” are easily retrieved and included in the SAN. Some clues of easier guillotines in BOARD_GAME allow the human players to solve the games by simply exploiting common sense or lin-

guistic knowledge but, as they typically refer to general concepts, they cause the system to build larger SANs (and CSLs) which overwhelm the ranking algorithms. As an example, consider the clues: *writer*, *city*, *castle*, *ship*, *story*. The solution for this game is *ghost*, which is quite easy to find for a human, but none of the proposed methods gave the correct answer, probably because clues are commonly used words having low discriminative power. Furthermore, the fact that all methods performed better on BOARD_GAME than on TV_GAME confirms the insight that guillotines in TV_GAME are more difficult to solve. The precision of Hybrid-CSL, $P = 0.056$ on both datasets, is obtained for $h = 10, t = 0.4$ and 10 entities retrieved from Freebase for each clue. This method outperformed the baseline only on TV_GAME: Freebase entities improve the reranking of the CSL whether more specific knowledge is required to solve the game. The other method that exploits Freebase entities, i.e., Freebase-CSL, achieves very low precision on both datasets ($P = 0.004$ on TV_GAME, $P = 0.007$ on BOARD_GAME, obtained with 10 entities), which is significantly under the baseline. In general, we observed that the knowledge stored in our CU repository is more effective than Freebase, maybe because in our approach we selected specific knowledge sources that would help a human solve the game, while Freebase is more general-purpose. Also the precision of the Query Expansion method is below the baseline ($P = 0.011$ on TV_GAME, obtained with 5 CUs, $P = 0.013$ on BOARD_GAME, obtained with 10 CUs). The poor precision is not due to the knowledge repository, since it is the same as the baseline, but depends on

Fig. 15. $P@k$ on the TV_GAME dataset.Fig. 16. $P@k$ on the BOARD_GAME dataset.

the algorithm adopted. All methods using spreading activation over the SAN (AL, RP, ERP, Hybrid-CSL) are always better in discovering keywords related to clues through CUs, than the others which leverage statistical methods to assess correlation between words (Query Expansion and Freebase-CLS).

$P@k$ of all compared methods on both datasets is reported in Figs. 15 and 16. ERP is the best performing method. Even short lists, containing top-5 and top-10 candidate solutions, achieve high precision: $P@5 = 0.398$ and $P@10 = 0.443$ on TV_GAME, while $P@5 = 0.433$ and $P@10 = 0.493$ on BOARD_GAME. In general, good performance by ERP and RP is observed on both datasets. They outperformed the AL

baseline, as well as the hybrid method using Freebase. We can conclude that reranking by using CUs definitely improves AL and is better than reranking which exploits Freebase entities.

V. FINAL REMARKS

This paper describes OTTHO, an artificial player for a complex language game, requiring human-level intelligence to be solved. The “brain” of the system is a spreading activation algorithm which links the clues of the game with the words within the knowledge repository, leading to a list of candidate solutions of the game. Algorithms for the reranking of the candidate solutions have been proposed and evaluated, with the aim

of selecting the word which is likely the *unique* answer of the game. Experiments compared the performance of the system with that of human players and other strategies using either a different knowledge repository or a different algorithm for producing candidate solutions. The main findings are:

- the proposed reranking methods outperformed human players on hard games (whose clues are difficult to interpret), and performed similarly to humans on games which can be more easily solved by the average player. This led us to conclude that, in general, the performance of the artificial player is comparable to that of average human players;
- the knowledge repository used to feed the artificial player matters. More specifically, experiments showed that using a general purpose knowledge repository, such as Freebase, causes a decrease of performance with respect to using the CU repository we designed.

The system has a great potential for applications outside of word games. For example, it could be adopted as a query expansion mechanism for Information Retrieval systems, as argued by Carpineto *et al.* [25]. In this scenario, terms in the seed query would be the clues provided to OTTHO, and expansion terms would be keywords selected from the CSL, as shown in Section IV-B-2. Even though we showed that the proposed query expansion method is not accurate in solving the game, experiments on datasets for the general query expansion task are needed to test its effectiveness in that context. The advantage would be the definition of a conceptual, rather than a statistical, correlation among expansion terms and query terms.

Another possible application is computing semantic relatedness of natural language texts, a task which requires access to vast amounts of common-sense and domain-specific world knowledge. Methods that represent the meaning of texts in a high-dimensional space of concepts derived from Wikipedia [17] have been already proposed. Our system might implement a strategy for computing text relatedness based on a richer knowledge repository than just Wikipedia. Given a document d_i , some keywords describing d_i could be provided to OTTHO (for example, the most significant terms can be obtained by applying feature selection), and keywords in the resulting CSL could be used to compute text similarity among d_i and another document d_j . The advantage would be that the correlation might actually reflect some “hidden associations” between d_i and d_j discovered by means of the keywords suggested by OTTHO, rather than using a classical similarity score based on a statistical correlation among terms occurring in both documents.

ACKNOWLEDGMENT

The authors would like to thank P. Tomeo for his help in the development of the reranking methods.

REFERENCES

- [1] M. L. Littman, “Review: Computer language games,” in *Proc. Comput. Games, 2nd Int. Conf., Rev. Papers*, T. A. Marsland and I. Frank, Eds., 2000, vol. 2063, pp. 396–404, ser. LNCS, Springer.
- [2] S. K. Lam, D. M. Pennock, D. Cosley, and S. Lawrence, “1 Billion Pages = 1 Million Dollars? Mining the web to play ‘Who wants to be a millionaire?’,” in *Proc. 19th Conf. Uncert. Artif. Intell.*, C. Meek and U. Kjærulff, Eds., 2003, pp. 337–345.
- [3] P. Molino, P. Basile, C. Santoro, P. Lops, M. de Gemmis, and G. Semeraro, M. Baldoni, C. Baroglio, G. Boella, and R. Micalizio, Eds., “A virtual player for ‘who wants to be a millionaire?’ based on question answering,” in *Proc. AI*IA 2013: Adv. Artif. Intell.—13th Int. Conf. Italian Assoc. Artif. Intell.*, 2013, vol. 8249, pp. 205–216, ser. Lecture Notes in Comput. Sci..
- [4] D. Ferrucci, E. Brown, J. Chu-Carroll, J. Fan, D. Gondek, A. A. Kalyanpur, A. Lally, J. W. Murdock, E. Nyberg, J. Prager, N. Schlaepfer, and C. Welty, “Building Watson: An overview of the DeepQA project,” *AI Mag.*, vol. 31, no. 3, pp. 59–79, 2010.
- [5] J. Chu-Carroll, E. W. Brown, A. Lally, and J. W. Murdock, “Identifying implicit relationships,” *IBM J. Res. Develop.*, vol. 56, no. 3, p. 12, 2012.
- [6] M. L. Littman, G. A. Keim, and N. Shazeer, “A probabilistic approach to solving crossword puzzles,” *Artif. Intell.*, vol. 134, pp. 23–55, 2002.
- [7] G. Angelini, M. Ermandes, and M. Gori, “Solving Italian Crosswords Using the Web,” in *AI*IA 2005: Adv. Artif. Intell.*, ser. Lecture Notes in Artificial Intelligence, S. Bandini and S. Manzoni, Eds. New York, NY, USA: Springer-Verlag, 2005, vol. 3673, pp. 393–405.
- [8] M. Ermandes, G. Angelini, and M. Gori, “A web-based agent challenges human experts on crosswords,” *AI Mag.*, vol. 29, no. 1, pp. 77–90, 2008.
- [9] M. Ermandes, G. Angelini, and M. Gori, “Webcrow: A web-based system for crossword solving,” in *Proc. 20th Nat. Conf. Artif. Intell. 17th Innov. Appl. Artif. Intell. Conf.*, M. M. Veloso and S. Kambhampati, Eds., 2005, pp. 1412–1417.
- [10] G. Semeraro, M. de Gemmis, P. Lops, and P. Basile, “An artificial player for a language game,” *IEEE Intell. Syst.*, vol. 27, no. 5, pp. 36–43, Sep.–Oct. 2012.
- [11] G. Semeraro, P. Lops, P. Basile, and M. de Gemmis, “On the tip of my thought: Playing the Guillotine game,” in *Proc. 21st Int. Joint Conf. Artif. Intell.*, 2009, pp. 1543–1548, Morgan Kaufmann.
- [12] M. Craven, D. DiPasquo, D. Freitag, A. McCallum, T. M. Mitchell, K. Nigam, and S. Slattery, “Learning to construct knowledge bases from the World Wide Web,” *Artif. Intell.*, vol. 118, no. 1–2, pp. 69–113, 2000.
- [13] R. Kosala and H. Blockeel, “Web mining research: A survey,” *SIGKDD Explorat.*, vol. 2, no. 1, pp. 1–15, 2000.
- [14] S. C. Deerwester, S. T. Dumais, T. K. Landauer, G. W. Furnas, and R. A. Harshman, “Indexing by Latent Semantic analysis,” *JASIS*, vol. 41, no. 6, pp. 391–407, 1990.
- [15] W. Buntine, S. Perttu, and V. Tuulos, “Using discrete PCA on Web pages,” in *Proc. Workshop Statist. Approaches Web Mining EACL/PKDD*, M. Gori, M. Ceci, and M. Nanni, Eds., 2004, pp. 99–110.
- [16] E. Gabrilovich and S. Markovitch, “Computing Semantic relatedness using Wikipedia-based explicit Semantic analysis,” in *Proc. 20th Int. Joint Conf. Artif. Intell.*, M. M. Veloso, Ed., 2007, pp. 1606–1611.
- [17] E. Gabrilovich and S. Markovitch, “Wikipedia-based Semantic interpretation for natural language processing,” *J. Artif. Intell. Res.*, vol. 34, no. 1, pp. 443–498, 2009.
- [18] C. Bizer, J. Lehmann, G. Kobilarov, S. Auer, C. Becker, R. Cyganiak, and S. Hellmann, “DBpedia—A crystallization point for the Web of data,” *Web Semantics: Sci., Serv. Agents World Wide Web*, vol. 7, no. 3, pp. 154–165, 2009.
- [19] F. M. Suchanek, G. Kasneci, and G. Weikum, “YAGO: A large ontology from Wikipedia and WordNet,” *Web Semantics Sci. Serv. Agents on the World Wide Web*, vol. 6, no. 3, pp. 203–217, 2008.
- [20] R. Baeza-Yates and B. Ribeiro-Neto, *Modern Information Retrieval*. Reading, MA, USA: Addison-Wesley, 1999.
- [21] J. R. Anderson, “A spreading activation theory of memory,” *J. Verbal Learn. Verbal Behav.*, vol. 22, pp. 261–295, 1983.
- [22] C. D. Manning and H. Schütze, *Foundations of Statistical Natural Language Processing*. Cambridge, MA, USA: MIT Press, 2001.
- [23] G. Salton, A. Wong, and C. S. Yang, “A vector space model for automatic indexing,” *Commun. ACM*, vol. 18, no. 11, pp. 613–620, 1975.
- [24] A. M. Collins and E. F. Loftus, “A spreading activation theory of semantic processing,” *Psychol. Rev.*, vol. 82, no. 6, pp. 407–428, 1975.
- [25] C. Carpineto and G. Romano, “A survey of automatic query expansion in information retrieval,” *ACM Comput. Surv.*, vol. 44, no. 1, p. 1, 2012.
- [26] C. Carpineto, G. Romano, and V. Giannini, “Improving retrieval feedback with multiple term-ranking function combination,” *ACM Trans. Inf. Syst.*, vol. 20, no. 3, pp. 259–290, 2002.
- [27] S. Royce and B. Straits, *Approaches to Social Research*, 3rd ed. New York: Oxford Univ. Press, 1999.



Pierpaolo Basile received the Ph.D. degree in 2009 with a dissertation on “Word Sense Disambiguation and Intelligent Information Access.”

He is a Postdoctoral Researcher in computer science at the University of Bari Aldo Moro, Italy. His research interests include word sense disambiguation, distributional semantics, information retrieval, and personalized information filtering.



Pasquale Lops received the Ph.D. degree in 2005 with a dissertation on “Hybrid Recommendation Techniques based on User Profiles.”

He is an Assistant Professor of Computer Science at the University of Bari Aldo Moro, Italy. His research interests include recommender systems, machine learning, user modeling, and information filtering.



Marco de Gemmis received the Ph.D. degree in 2005 with a dissertation on “Learning User Profiles from Text for Personalized Information Access.”

He is Assistant Professor of Computer Science at the University of Bari Aldo Moro, Italy. His research interests include natural-language processing, machine-learning techniques for text categorization, information retrieval, and personalized information filtering.



Giovanni Semeraro received the M.Sc. degree in computer science from the University of Bari Aldo Moro, Italy,

He is currently an Associate Professor of Computer Science at the University of Bari Aldo Moro, and teaches “Intelligent information access,” “Natural language processing,” and “Programming languages.” He leads the Semantic Web Access and Personalization (SWAP) Research Group. He has been a Visiting Scientist with the Department of Information and Computer Science, University of California at Irvine, in 1993. From 1989 to 1991, he was a researcher at Tecnopolis CSATA Novus Ortus, Bari, Italy. His research interests include AI; recommender systems; language games; intelligent information mining, retrieval, and filtering; semantic and social computing; machine learning; natural language processing; the semantic web; and personalization.